МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №6
Варіант 4

на тему «Замикання та декоратори як функції вищого порядку»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. навчитися створювати замикання, що утримують стан; писати параметризовані декоратори як фабрики декораторів; використовувати functools.wraps, ParamSpec і ін'єкцію залежностей для тестованості.

Теоретичні відомості

Замикання — функція, що «пам'ятає» змінні зовнішньої області видимості навіть після її завершення. Для зміни захопленої змінної використовують **nonlocal**.

Декоратор — функція вищого порядку decorator(func) → wrapper. **Параметризований декоратор** — фабрика factory(*opts) → decorator → wrapper; параметри передаються в обгортку через замикання. **@functools.wraps** зберігає __name__, __doc__ і додає __wrapped__.

Для типізації використовують ParamSpec і TypeVar (a Concatenate — для додавання аргументу). Залежності (logger, clock, sleep) передаються як Callable, щоб тести були детермінованими.

Індивідуальне завдання (варіант 4)

Тема «Авторизація користувачів». Компоненти: closure для списку ролей, параметризований @require_role, @timed. Дозволяти виклик функції лише користувачам потрібної ролі. Вимоги: @wraps, власний виняток, логер як залежність.

Хід роботи

1. Замикання

make_counter зберігає стан через nonlocal; make_role_checker замикає набір дозволених ролей (копію у frozenset); make_session зберігає поточного користувача.

```
def make_counter(start: int = 0) -> Callable[[], int]:
    """Кожен виклик повертає наступне значення; стан ``value`` живе в замиканні."""
    value = start

    def inc() -> int:
        nonlocal value
        value += 1
        return value

    return inc

def make_role_checker(
    allowed: Iterable[str], *, mode: str = "any"
) -> Callable[[Iterable[str]], bool]:
    """Замикання над дозволеними ролями.

    mode="any" — достатньо однієї з ролей; mode="all" — потрібні всі.
    """
```

```
    allowed_set = frozenset(allowed)
    if mode not in ("any", "all"):
        raise ValueError("mode must be 'any' or 'all'")

    def check(user_roles: Iterable[str]) -> bool:
        roles = frozenset(user_roles)
        if mode == "all":
            return allowed_set <= roles
        return bool(allowed_set & roles)

    return check
```

Лістинг 1 — фрагмент файлу closures.py

2. Власний виняток і декоратор @require_role

AuthorizationError — підклас PermissionError з полями user, required, func_name. require_role підтримує режими any / all, ієрархію ролей і логер-залежність; при відмові тіло функції не виконується.

```
class AuthorizationError(PermissionError):
    """Власний виняток: у користувача немає потрібної ролі."""

    def __init__(self, user: str, required: frozenset[str], func_name: str) -> None:
        self.user = user
        self.required = required
        self.func_name = func_name
        need = ", ".join(sorted(required))
        super().__init__(f"user '{user}' needs role [{need}] to call {func_name}()")

def require_role(
    *roles: str,
    mode: Literal["any", "all"] = "any",
    logger: Logger = _silent,
    expand: RoleExpander | None = None,
) -> Callable[[Callable[Concatenate[User, P], R]], Callable[Concatenate[User, P], R]]:
    """Фабрика декоратора: дозволяє виклик лише користувачу з роллю ``roles``.
    Перший позиційний аргумент декорованої функції — ``User``.
    ``expand`` — політика ієрархії ролей (наприклад, admin включає editor).
    """
    if not roles:
        raise ValueError("at least one role is required")
    required = frozenset(roles)
    check = make_role_checker(required, mode=mode)  # замикання над ролями

    def decorator(
        func: Callable[Concatenate[User, P], R],
    ) -> Callable[Concatenate[User, P], R]:
        @functools.wraps(func)
        def wrapper(user: User, /, *args: P.args, **kwargs: P.kwargs) -> R:
            effective = expand(user.roles) if expand else user.roles
            if not check(effective):
                logger(f"DENY {user.name} -> {func.__name__}")
                raise AuthorizationError(user.name, required, func.__name__)
            logger(f"ALLOW {user.name} -> {func.__name__}")
            return func(user, *args, **kwargs)

        return wrapper

    return decorator
```

Лістинг 2 — фрагмент файлу decorators.py

3. Декоратори @timed та @logged

timed вимірює час з ін'єктованим clock і пише лог у блоці finally (навіть при винятку); logged журналює аргументи, результат або виняток.

```
def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Logger = print,
    clock: Callable[[], float] = time.perf_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Вимір часу виконання; лог пишеться навіть якщо функція кинула виняток."""
    factor = _FACTORS[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            start = clock()
            try:
                return func(*args, **kwargs)
            finally:
                elapsed = (clock() - start) * factor
                logger(f"[timed] {name} took {elapsed:.3f} {unit}")

        return wrapper
    return decorator
```

Лістинг 3 — фрагмент файлу decorators.py

4. Порядок і композиція декораторів

Якщо @timed зовнішній, вимірюються й заборонені спроби; якщо внутрішній — таймер не запускається. Це перевіряють тести. compose_decorators(a, b)(f) = a(b(f)).

```
def test_timed_outside_measures_denied_calls_too() -> None:
    sink = make_memory_sink()

    @timed("op", logger=sink.write, clock=ticking_clock())
    @require_role("admin")
    def op(user: User) -> str:
        return "ok"

    with pytest.raises(AuthorizationError):
        op(GUEST)
    assert sink.lines() == ("[timed] op took 500.000 ms",)  # спроба теж виміряна
def test_timed_inside_is_skipped_when_denied() -> None:
    sink = make_memory_sink()

    @require_role("admin")
    @timed("op", logger=sink.write, clock=ticking_clock())
    def op(user: User) -> str:
        return "ok"

    with pytest.raises(AuthorizationError):
        op(GUEST)
    assert sink.lines() == ()  # перевірка ролі зупинила виклик ДО таймера
    assert op(ADMIN) == "ok"
    assert len(sink.lines()) == 1
```

Лістинг 4 — фрагмент файлу tests/test_composition.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
=== require_role + ієрархія ролей ===
[AUDIT] ALLOW alice -> view_report
[PERF] [timed] view_report took 0.008 ms
  OK   alice.view_report: alice переглядає звіт 'Q3'
[AUDIT] ALLOW bob -> view_report
[PERF] [timed] view_report took 0.005 ms
  OK   bob.view_report: bob переглядає звіт 'Q3'
[AUDIT] ALLOW eve -> view_report
[PERF] [timed] view_report took 0.003 ms
  OK   eve.view_report: eve переглядає звіт 'Q3'
[AUDIT] ALLOW bob -> create_document
  OK   bob.create_document: 101
[AUDIT] ALLOW alice -> create_document
  OK   alice.create_document: 102
[AUDIT] DENY eve -> create_document
  FAIL eve.create_document: AuthorizationError: user 'eve' needs role [editor] to call
      create_document()
=== тільки admin + @logged ===
[AUDIT] WARNING call delete_document(User(name='alice', roles=frozenset({'admin'})), 101)
[AUDIT] ALLOW alice -> delete_document
[AUDIT] WARNING delete_document -> 'документ #101 видалено користувачем alice'
  OK   alice.delete_document: документ #101 видалено користувачем alice
[AUDIT] WARNING call delete_document(User(name='bob', roles=frozenset({'editor'})), 101)
[AUDIT] DENY bob -> delete_document
[AUDIT] WARNING delete_document raised AuthorizationError: user 'bob' needs role [admin] to
    call delete_document()
  FAIL bob.delete_document: AuthorizationError: user 'bob' needs role [admin] to call
      delete_document()
=== mode='all' ===
[AUDIT] DENY alice -> rotate_keys
  FAIL alice.rotate_keys: AuthorizationError: user 'alice' needs role [admin, security] to
      call rotate_keys()
[AUDIT] ALLOW root -> rotate_keys
  OK   root.rotate_keys: ключі оновлено

=== роль із сесії ===
[AUDIT] ANON -> reset_cache
  FAIL anonymous.reset_cache: AuthenticationError: login required for reset_cache()
[AUDIT] DENY bob -> reset_cache
  FAIL bob(session).reset_cache: AuthorizationError: user 'bob' needs role [admin] to call
      reset_cache()
[AUDIT] ALLOW alice -> reset_cache
  OK   alice(session).reset_cache: кеш скинуто
  входів у сесії: 2
=== метадані після декорування ===
  __name__   : delete_document
  __doc__    : Видалити документ (тільки admin).
  __wrapped__: True
```

Рисунок 1 — результат роботи програми (Lab_06)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_06
configfile: pyproject.toml
```

```
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 23 items
tests/test_closures.py::test_counter_keeps_state_in_closure PASSED       [  4%]
tests/test_closures.py::test_logger_with_injected_sink PASSED            [  8%]
tests/test_closures.py::test_role_checker_any_and_all PASSED             [ 13%]
tests/test_closures.py::test_role_checker_copies_roles PASSED            [ 17%]
tests/test_closures.py::test_role_hierarchy_is_transitive PASSED         [ 21%]
tests/test_closures.py::test_role_hierarchy_handles_cycles PASSED        [ 26%]
tests/test_closures.py::test_session_state PASSED                        [ 30%]
tests/test_closures.py::test_user_is_immutable_and_hashable PASSED       [ 34%]
tests/test_composition.py::test_timed_outside_measures_denied_calls_too PASSED [ 39%]
tests/test_composition.py::test_timed_inside_is_skipped_when_denied PASSED [ 43%]
tests/test_composition.py::test_compose_decorators_equals_stacking PASSED [ 47%]
tests/test_composition.py::test_logged_outside_logs_authorization_error PASSED [ 52%]
tests/test_decorators.py::test_require_role_allows_and_denies PASSED     [ 56%]
tests/test_decorators.py::test_denied_function_body_is_not_executed PASSED [ 60%]
tests/test_decorators.py::test_require_role_any_vs_all_mode PASSED       [ 65%]
tests/test_decorators.py::test_require_role_with_hierarchy PASSED        [ 69%]
tests/test_decorators.py::test_require_role_needs_at_least_one_role PASSED [ 73%]
tests/test_decorators.py::test_require_session_role PASSED               [ 78%]
tests/test_decorators.py::test_timed_uses_injected_clock PASSED          [ 82%]
tests/test_decorators.py::test_timed_logs_even_on_exception PASSED       [ 86%]
tests/test_decorators.py::test_timed_units PASSED                        [ 91%]
tests/test_decorators.py::test_logged_result_and_exception PASSED        [ 95%]
tests/test_decorators.py::test_wraps_preserves_metadata PASSED           [100%]
============================== 23 passed in 0.06s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_06)

Висновок

Реалізовано систему авторизації на основі замикань і параметризованих декораторів. Замикання зберігають набір ролей, ієрархію ролей, лічильник і стан сесії (nonlocal). Декоратори require_role, require_session_role, timed і logged використовують functools.wraps, ParamSpec / TypeVar / Concatenate і отримують логер та годинник як залежності, тому 23 тести виконуються детерміновано без реальних затримок. Показано, що порядок декораторів впливає на поведінку.